

一、SDR 内存介绍

OCAP平台上内存可划分为TCM，DDR，SHRAM，TOP RAM四类

1. TCM

仅仅存在于DSP核，分为DTCM和ITCM两类，DTCM为片内数据存储器，ITCM为片内指令存储器。一般情况下，TCM所属的处理器对TCM进行访问无延迟。

例如：

```
#define WAVEFORM_ITCM_SECTION      __attribute__((section(".CSECT CODE_TCM_WAVEFORM_A_SECTION")))
```

Isf 链接脚本把代码/数据“划片”到几块指定物理地址（DTCM、DDR、SHRAM1），而 **attribute**((section(...))) 的宏是给编译器打标签，告诉“这个变量/函数属于哪个段名（section）”，最终由链接脚本把同名段放进对应地址。

2. SHRAM

位于SoC内部，主要在CP子系统内部使用，可以配置为数据存储空间或指令存储空间。对SHRAM的访问延迟高于各处理器对TCM的访问，但低于TOP RAM和DDR。

3. TOP RAM

处于Soc内部，设计用于Top子系统内部使用，以及用于TL420处理器和AP处理器之间进行数据交换的用途。目前TopRAM的空间由OCAP软件平台完全控制，波形用户无法使用Top RAM。

4. DDR

处于处理器外部的通用存储器，在整个SoC内部共享，既可以存放指令也可以存放数据，访问时间是所有四种内存中最高的。

5. LSF脚本

Isf 做了什么

- 定义了几类内存区域（classes）及地址/大小/缓存属性：
 - data_int_wf_a：内部 **DTCM 数据**，[0x0000A000 ~ 0x0000DFFF]，16KB
 - code_int_wf_a：内部 **ITCM/TCM 代码**，[0x00027000 ~ 0x0006FFFF]，约 292KB
 - data_ddr_wf_a： **DDR 数据**，[0x03600000 ~ 0x03EFFFFF]，9MB，可缓存
 - data_shram3_wf_a： **SHRAM3 可缓存数据**，[0x81940000 ~ 0x8195FFFF]
 - data_shram3_wf_a_noncache： **SHRAM3 非缓存数据**，[0x81A00000 ~ 0x81A27BFF]
 - code_ddr_wf_a： **DDR 代码**，[0x02000000 ~ 0x023FFFFF]
 - code_shram1_wf_a： **SHRAM1 代码**，[0x81040000 ~ 0x81062BFF]
 - 在这些类下，把具体段名放到确定地址（at 0x...）或按顺序紧邻布局（next）：
 - DATA_TCM_WAVEFORM_A_SECTION 放到 0x0000A000（也就是 DTCM 数据起始处）
 - CODE_TCM_WAVEFORM_A_SECTION 放到 0x00027000
 - DATA_SHRAM3_WAVEFORM_A_SECTION 放到 0x81940000
 - DATA_SHRAM3_FOR_CODING_RESULT_SECTION 放到 0x81950000 size 22000
- 等等

例如：

```
#define DATA_TCM_WAVEFORM_A_SECTION \
    __attribute__((section(".DSECT DATA_TCM_WAVEFORM_A_SECTION")))
```

然后对变量使用：

```
static uint8_t rx_buf[2048] DATA_TCM_WAVEFORM_A_SECTION;
```

即，这份 lsf 已经把 段名 → 物理地址/缓存属性映射定好。

section(...) 宏要用同一个段名把对象放进想要的那块内存。

二、内存池创建

D:\20220728\OCAP_V3.00.02.P03_dongda_20231020\app\TFWD\XC4210\submod\src\demo_main_xc4210.c

1. 函数原型

```
UINT16 osa_mem_pool_create(  
    IN UINT16 u16_pool_type,  
    IN VOID_PTR stp_pool_cfg  
);
```

参数解释：

- 1. u16_pool_type
 - 类型：UINT16
 - 作用：指定要创建的内存池类型。支持的类型有：
 - OSA_MEM_TYPE_NORMAL —— 普通内存池
 - OSA_MEM_TYPE_STATIC —— 静态内存池（由用户事先分配固定内存区域）
 - OSA_MEM_TYPE_SEMI_STATIC —— 半静态内存池（部分动态，部分固定）
 - OSA_MEM_TYPE_FATHER_BUFFER —— 父缓冲池（可能用于大块内存管理，子 buffer 从里面划分）
- 2. stp_pool_cfg
 - 类型：VOID_PTR，指向不同的配置结构体。
 - 作用：根据 u16_pool_type 的不同，需要传入不同的配置结构体，定义内存池的大小、数量等参数：
 - 对应 OSA_MEM_TYPE_NORMAL → osa_mem_dynamic_cfg_t
 - 对应 OSA_MEM_TYPE_STATIC → osa_mem_static_cfg_t
 - 对应 OSA_MEM_TYPE_SEMI_STATIC → osa_mem_semi_static_cfg_t
 - 对应 OSA_MEM_TYPE_FATHER_BUFFER → osa_mem_father_mbuf_cfg_t

返回值：

- 成功：返回一个 UINT16 类型的 内存池 ID（相当于句柄，后续用它来分配/释放内存）。
- 失败：返回 0xFFFF，表示内存池创建失败。

配置结构体 osa_mem_dynamic_cfg_t：

```
typedef struct osa_mem_dynamic_cfg_tag {
    VOID_PTR    vp_start_addr;        // 内存池起始地址
    VOID_PTR    vp_end_addr;          // 内存池结束地址
    SINT32      s32_pool_size;        // 内存池大小，单位字节，4字节对齐
    SINT32      s32_alloc_size_min;    // 最小分配单元（块大小下限）
    SINT32      s32_alloc_size_max;    // 最大分配单元（块大小上限）
    UINT16      u16_alloc_step;        // 分配步进，决定分配粒度
    UINT16      u16_reserved;          // 保留字段
} osa_mem_dynamic_cfg_t;
```

调用：

```
g_u16_spool_ID[0] = OSA_MEM_POOL_CREATE(OSA_MEM_TYPE_NORMAL, &st_pool_cfg);
```

查看配置文件 wf_config.ini 可以查看内存池的位置和大小

此处以 XC4210 为例：

项目	名称	类型说明	RAM属性值	大小 (Byte)	其它属性	含义
ram0	GPOOL0	General Pool	0	20480	0	20KB，通用内存池
ram1	GPOOL1	General Pool	1	20480	0	20KB，通用内存池
ram2	GPOOL2	General Pool	2	4096	0	4KB，通用内存池
ram3	SPOOL0	Stack Pool	0	16384	0	16KB，任务栈池
ram4	SPOOL1	Stack Pool	2	20480	0	20KB，任务栈池

三、定义线程列表

D:\20220728\OCAP_V3.00.02.P03_dongda_20231020\app\TFWD\XC4210\submod\src\demo_main_xc4210.c

1. osa_task_create() 函数

任务创建接口，操作系统会根据传入的参数（任务名、优先级、入口函数、栈大小等）创建一个新任务（线程）。

下表 osa_task_create() 准备的一组参数。

2. 参数与任务描述表的对应关系

任务描述表字段	对应 osa_task_create() 参数	类型	说明
"42d_hls"	p_task_name	CHAR_PTR	任务名，最多 8 个字符
(UINT16)0x23	u16_priority	UINT16	任务优先级 (0~127)
(UINT32)0x0000	u32_task_param	UINT32	启动参数，会传递给 init/main 函数
demo_xc4210_task_init	fp_task_init	osa_task_init_t	初始化函数，任务启动时调用
demo_xc4210_task	fp_task_main	osa_task_main_t	主循环函数，任务执行体
demo_xc4210_task_delete	fp_task_delete	osa_task_delete_t	删除函数，释放资源

任务描述表字段	对应 osa_task_create() 参数	类型	说明
OSA_INVALID_U16_ID	u16_stack_pool_id	UINT16	栈池 ID，未使用时给无效值
16384	u16_stack_size	UINT16	栈大小，单位字节

3. 举例

以 "42d_h1s" 任务为例，传入 osa_task_create() 时就等价于：

这样系统就会创建一个名字为 "42d_h1s" 的任务，分配 16KB 栈，优先级 0x23，运行 demo_xc4210_task() 作为主循环。

for 循环就是把 **任务描述表** 和 osa_task_create() 串起来。

```
/* Create Demo Waveform Tasks */
for (u32_i = 0; u32_i < DEMO_TASK_AMT; u32_i++)
{
    OSA_TASK_CREATE(
        C_ST_OAL_APP_TASK_DESC_TBL[u32_i].task_name,
        C_ST_OAL_APP_TASK_DESC_TBL[u32_i].u16_priority,
        C_ST_OAL_APP_TASK_DESC_TBL[u32_i].u32_task_param,
        C_ST_OAL_APP_TASK_DESC_TBL[u32_i].fp_task_init,
        C_ST_OAL_APP_TASK_DESC_TBL[u32_i].fp_task_main,
        C_ST_OAL_APP_TASK_DESC_TBL[u32_i].fp_task_delete,
        DEMO_TASK_SPOOL_ID_1,
        C_ST_OAL_APP_TASK_DESC_TBL[u32_i].u16_stack_size
    );
}
```

- 1. 循环遍历所有任务
- 2. 逐个调用 OSA_TASK_CREATE()
- 3. 栈池 ID
 - 所有任务的 stack_pool_id 都为 DEMO_TASK_SPOOL_ID_1

四、消息机制

1. 创建消息队列

函数原型：

```
UINT16 osa_msg_queue_create(IN CONST_UINT16 u16_mb_size);
```

- 参数 u16_mb_size
 - 类型：UINT16
 - 含义：**消息块大小（单位：字节）**，即每条消息在队列中占用的存储空间大小。
 - 这里传入的是 C_U16_MSG_Q_LENGTH[u32_i]，说明系统里预先定义了一个数组，配置了每个消息队列的块大小。
- 返回值
 - 成功：返回一个消息队列 ID（UINT16），后续 osa_msg_send()、osa_msg_receive() 等函数都要用这个 ID 来操作。
 - 失败：返回 0xFFFF。

示例代码：

D:\20220728\OCAP_V3.00.02.P03_dongda_20231020\app\TFWD\XC4210\submod\src\demo_main_xc4210.c

```
g_u16_msg_qID[u32_i] = OSA_MSG_QUEUE_CREATE(C_U16_MSG_Q_LENGTH[u32_i]);
```

结合 `osa_msg_queue_create()` 的说明，其实是在 **创建一个消息队列**，并把返回的消息队列 ID 保存到 `g_u16_msg_qID[]` 里。

2. 消息接收

函数原型

```
osa_msg_t *osa_msg_receive(  
    IN CONST_UINT16 u16_queue_id,  
    IN CONST_osa_prim_id_t *stp_prim_id,  
    IN CONST_UINT32 u32_timeout  
);
```

返回值：

- 成功 → 返回收到的消息指针（`osa_msg_t*`）。
- 超时/失败 → 返回 `NULL_PTR`。

参数含义：

1. `u16_queue_id`

- 消息队列的 ID 编号。
- 在前面 `OSA_MSG_QUEUE_CREATE()` 时返回的值。

2. `stp_prim_id`

- 消息 ID 过滤器。
- 传入一个数组结构 {消息个数, 消息ID0, 消息ID1, ...}。
- 函数会在队列中查找符合这些 ID 的消息。
- 这里的 `st_wait_msg_list` 定义是：

```
osa_prim_id_t CONST st_wait_msg_list[2] = { (UINT32)1, (UINT32)OSA_MSG_ANY_ID };
```

- 第一个元素 1 → 表示有 1 个消息 ID 需要匹配。
- 第二个元素 `OSA_MSG_ANY_ID` → 表示接收**任意消息**。

3. `u32_timeout`

- 等待超时时间 (ms)。
- 宏定义可选：
 - `OSA_NO_WAIT` → 立即返回，不等待。
 - `OSA_WAIT_FOREVER` → 永久阻塞，直到有消息。
- 这里用 `OSA_WAIT_FOREVER` → 即：**任务会一直挂起，直到队列里有消息进来才会被唤醒。**

示例：

D:\20220728\OCAP_V3.00.02.P03_dongda_20231020\app\TFWD\XC4210\submod\src\demo_main_xc4210.c

```
stp_msg = osa_msg_receive(  
    DEMO_XC4210_TASK_MSG_Q,    // 队列 ID  
    st_wait_msg_list,          // 等待的消息 ID 列表  
    (UINT16)OSA_WAIT_FOREVER   // 永久等待  
);
```

3. 消息发送

函数原型：

```
OSA_STATUS osa_msg_send(  
    IN CONST_UINT16 u16_queue_id,    // 目标队列 ID  
    IN osa_msg_t *stp_msg,           // 消息指针  
    IN CONST_UINT32 u32_timeout     // 超时参数  
);
```

参数说明：

1. **u16_queue_id**
 - 目标消息队列的 ID 编号。
 - 必须是之前 `osa_msg_queue_create()` 创建时返回的 ID。
2. **stp_msg**
 - 消息的首地址（指针）。
 - 指向一个 `osa_msg_t` 类型结构体（消息头+消息体）。
 - 说明：发送方**不能再继续操作或释放该内存**，因为消息的所有权已经交给队列/接收方。
 - 接收方负责处理并释放消息。
3. **u32_timeout**
 - 发送超时时间，单位 ms。
 - 宏可选：
 - `OSA_NO_WAIT` → 立即返回，如果队列满就报错。
 - `OSA_WAIT_FOREVER` → 永远阻塞，直到队列有空位。
 - 或者指定具体超时时间。

返回值：

- **成功**： `OSA_SUCCESS`
- **失败**： `OSA_WAIT_TIMEOUT` （超时或队列满）

功能描述：

- 将消息 `stp_msg` **挂到指定队列的尾部**。
- 接收任务调用 `osa_msg_receive(queue_id, ...)` 就能取出这个消息。
 - 发送方： `osa_msg_send()`
 - 接收方： `osa_msg_receive()`

五、线程函数

1. 线程主循环

demo_xc4210_task() 主循环，“消息驱动任务”。核心流程如下：

1. 任务循环

```
while (1) { ... }
```

2. 阻塞等待消息

```
const osa_prim_id_t st_wait_msg_list[2] = {1, OSA_MSG_ANY_ID};

stp_msg = osa_msg_receive(DEMO_XC4210_TASK_MSG_Q, st_wait_msg_list, OSA_WAIT_FOREVER);
```

- 从消息队列 DEMO_XC4210_TASK_MSG_Q 无限期阻塞等待。
- st_wait_msg_list 语义是“等待 1 个消息 ID，且 ID 不限 (ANY)”，即来什么消息都收。

3. 解析消息参数

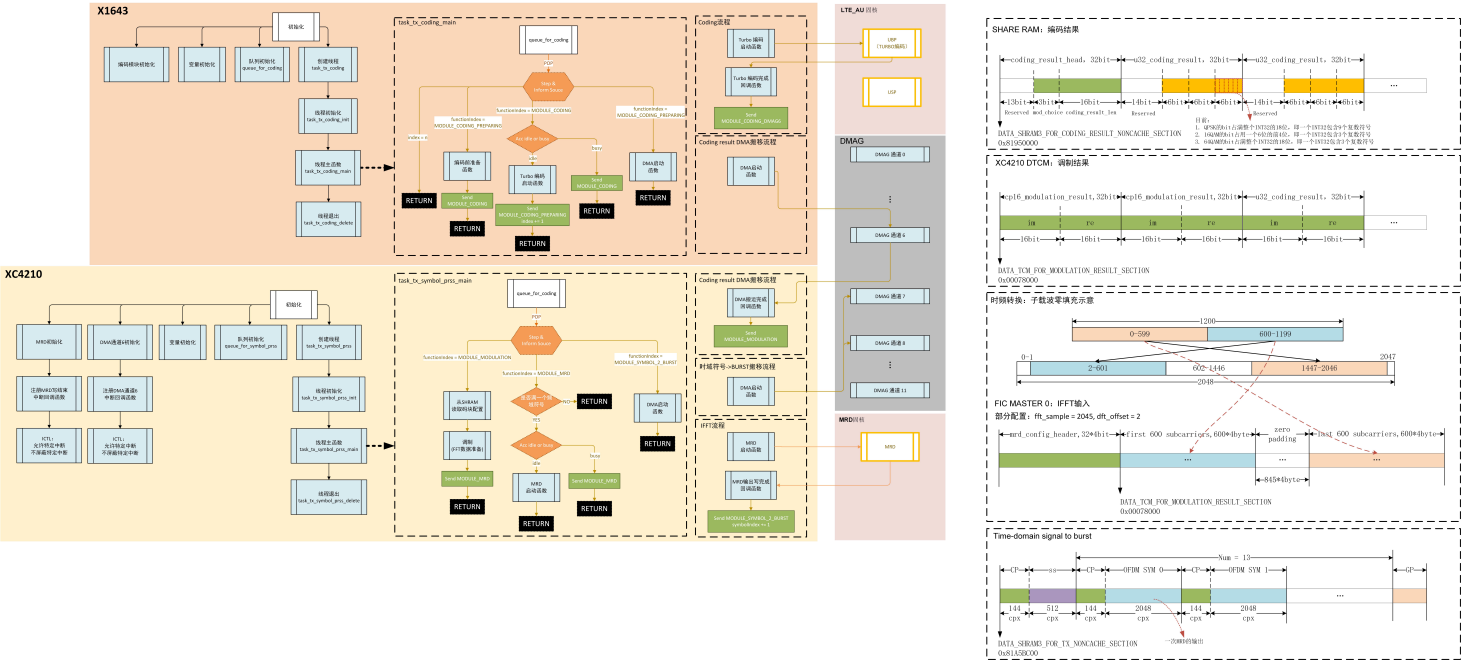
```
u16_core_id = ((demo_case_t*)FSM_PARAM_PTR(stp_msg))->u16_core_id;
u16_demo_cmd = ((demo_case_t*)FSM_PARAM_PTR(stp_msg))->u16_type_id;
```

- 消息体按 `demo_case_t` 解释，取出 `core_id` 与 `type_id`（命令码）。

4. 分支处理：当 `core_id==2 && type_id==xxxxx`

执行各个分支处理

六、发端结构



1. X1643 核

(a) 初始化

初始化 X1643 核心（或线程）的发射任务的入口，要完成了 消息队列准备、硬件中断注册、以及编码加速器的初始状态设置。

```
queue_for_coding = OSA_QUEUE_CREATE("queue_for_coding", 4, 16);

osa_task_create("task_tx_coding", (UINT16)55, (UINT32)100,
    task_tx_coding_init, task_tx_coding_main, task_tx_coding_delete,
    DEMO_TASK_SPOOL_ID_1, (UINT16)2048);
```

1. 创建队列

- queue_for_coding 用来存放发端需要编码的数据任务，队列深度是 4，单个消息大小 16 字节。
- 相当于在 X1643 编码任务 和其他模块之间建立了通信通道。

2. 创建任务

- task_tx_coding：这是负责执行 编码处理相关 的主任务，运行时会不断从 queue_for_coding 里取任务，调用硬件加速器去做编码，然后把结果推送到 XC4210。

此时，构造一条“编码准备（符号0）”的任务消息，投递到编码队列 queue_for_coding，以开启整个线程。

(b) 线程主函数

这是 X1643 侧的“编码线程主函数”，采用“队列 + 状态机”的方式驱动一帧里各个 OFDM 符号的编码与搬移。核心思路：不停从 queue_for_coding 取消息 → 根据 functionIndex 分支处理 → 必要时把消息改写后再投回同一个队列，形成流水线。

下面按代码路径拆解：

入口与主循环：

```
OSA_STATUS task_tx_coding_main(){
    SymbolStruct msg_receive;
    ...
    while(1){
        ret = osa_queue_receive(queue_for_coding, (VOID**)&msg_receive, OSA_WAIT_FOREVER);
        ...
    }
}
```

- 线程一直阻塞在 queue_for_coding 上，取到一条消息 msg_receive 才继续。
- SymbolStruct 为接收消息结构体

计时标记（用于性能分析）

```
if (msg_receive.index == 15){
    u32_time_start[99] = *pCP_TIMER3_TCV;
}
```

- 当处理到子帧最后一个符号时，不作其它处理，打一个时间戳用于统计整帧耗时。

阶段一：MODULE_CODING_PREPARING（编码准备）

```
else if (msg_receive.functionIndex == MODULE_CODING_PREPARING){
    coding_prepare_fun();           // 做编码前准备（装参、缓冲区、门限等）
    if (startCoding == 0){
        startCoding = 1;
        u32_time_start[0] = *pCP_TIMER3_TCV; // 启动总起点计时
        msg_receive.functionIndex = MODULE_CODING;
        osa_queue_send(queue_for_coding, &msg_receive, OSA_WAIT_FOREVER);
    }
}
```

- 首次进入时：设置 startCoding=1 并记录总体起点计时。
- 然后把同一条消息改成 MODULE_CODING 再次投回队列，驱动进入下一阶段（TURBO 编码）。
- 之后每次准备阶段通常也是为了给下一次编码做铺垫（形成“准备 ↔ 编码”的乒乓）。

阶段二：MODULE_CODING（调用编码加速器）

```
else if (msg_receive.functionIndex == MODULE_CODING){
    if (coding_acc_idle_or_not == COING_ACC_IDLE){
        short modulation_choice = 0;
        short coding_result_len = 2112;           // 编码后比特数（示例）
        short coding_result_int_len = 118;        // 以 int(4B) 为单位的长度
        coding_result_head_t *coding_result_head = NULLPTR;

        u32_time_start[2 + msg_receive.index] = *pCP_TIMER3_TCV; // 每个符号的编码起点计时

        // 启动编码（硬件加速器）
        algo_hac_brp_case_for_6bit_youhua(temp, 824, 2, msg_receive.index, coding_result_int_len);

        coding_result_head->modulation_choice = modulation_choice;
        coding_result_head->coding_result_len = coding_result_len;
        u32_coding_result[(coding_result_int_len+1)*msg_receive.index] = *(int *)coding_result_head;

        // 下一步回到准备阶段
        msg_receive.functionIndex = MODULE_CODING_PREPARING;
        osa_queue_send(queue_for_coding, &msg_receive, OSA_WAIT_FOREVER);
    } else {
        // 加速器忙：把同一条消息塞回队列，稍后重试
        osa_queue_send(queue_for_coding, &msg_receive, OSA_WAIT_FOREVER);
        osa_print(1, "coding acc busy happened!");
    }
}
```

- 只有当 coding_acc_idle_or_not 表示空闲时才调用编码
- 如果忙就重投队列

阶段三：MODULE_CODING_DMAG6（编码结果 DMA 搬移到 XC4210）

```
else if (msg_receive.functionIndex == MODULE_CODING_DMAG6){
    int *dst_add = (msg_receive.index & 1) ? pHAC_LTE_SUBF_SPRAM_B_ADDR
                                           : pHAC_LTE_SUBF_SPRAM_A_ADDR;

    u32_time_start[20 + msg_receive.index] = *pCP_TIMER3_TCV; // 每个符号的搬移起点计时

    dmag_mem_copy_via_ch6_for_coding_result_send_2_XC4210(
        u32_coding_result + (msg_receive.reserve_1+1)*msg_receive.index + 1, // 源：跳过头 1 int
        dst_add, // 目的：A/B 两块 ping-pong
        msg_receive.reserve_1 * 4 // 长度（字节），reserve_1 是“数据 int 数”
    );

    msg_receive.functionIndex = MODULE_CODING; // 回到编码阶段，准备下一个符号
    msg_receive.index += 1; // 符号索引自增
    osa_queue_send(queue_for_coding, &msg_receive, OSA_WAIT_FOREVER);
}
```

- 根据奇偶把目标 SPRAM 切到 A/B（ping-pong）区域，利于流水线并行。
- 计时： u32_time_start[20+index] 记录每个符号的 DMA 搬移起点。
- 完成后把同一条消息改回 MODULE_CODING 并把 index++，继续处理下一个符号。

(c) 编码 ISR

编码加速器的中断回调（ISR），用来把“编码完成”的事件接到编码线程状态机里去，驱动下一步 DMA 搬移。

编码完成后，触 coding_acc_callback 回调函数。

2. XC4210 核

(a) 初始化

init_XC4210_task_for_tx() 是XC4210 侧“发射（TX）任务”的初始化。

把DMA通道中断、MRD中断、编码/调制数据缓冲区与状态全部就位，确保 XC4210 的 TX 线程能正确接收来自 X1643 的编码结果、触发后续搬移/调制/MRD 写入流水线。

init_task_queue_for_4210_for_tx() 和 X1643初始化函数类似，是 XC4210 平台发射端的任务队列初始化函数：

1. 创建消息队列 queue_for_symbol_prss：
用来存放与符号处理相关的消息。
2. 创建符号处理任务线程 task_tx_symbol_prss：
 - init：做资源分配、变量初始化。
 - main：循环从 queue_for_symbol_prss 里取消息，执行相应的符号处理操作。
 - delete：销毁线程时清理资源。

(b) 线程主函数

XC4210 侧“符号处理”线程的主循环：（task_tx_symbol_prss_main）。

它从 queue_for_symbol_prss 收消息，根据 functionIndex 驱动三个阶段：

- 调制输出累计→送 MRD 加速器
- 把编码比特调制成复数符号
- 把完整符号拼成突发（添加CP并搬移）

入口与主循环：

- 一直阻塞在 queue_for_symbol_prss 上拿消息：osa_queue_receive(..., OSA_WAIT_FOREVER)
- msg_receive.functionIndex 决定走哪条处理路径。

A) MODULE_MRD：判断是否凑够一个符号并送 MRD

```
short sym_wait_to_MRD = (short)(cur_modulation_result_p - cur_mrd_acc_in_p);
if (sym_wait_to_MRD >= 1200) {
    if (MRD_acc_idle_or_not == MRD_ACC_IDLE) {
        // 送 MRD: 把 1200 个调制符号封装/写入到 tx_data_prepare_p (含头/对齐/3DDMA)
        mrd_accelerator_genHeadDirect_2045sample_3ddma(cur_mrd_acc_in_p, tx_data_prepare_p, 0, 1, 0);
        // 推进目的与源指针 (2192=2048数据+144CP; 源指针推进1200个调制符号)
        tx_data_prepare_p += 2192;
        cur_mrd_acc_in_p += 1200;
    } else {
        // MRD 忙: 把消息丢回队列等待下次再试
        osa_queue_send(queue_for_symbol_prss, &msg_receive, OSA_WAIT_FOREVER);
    }
} else {
    // 未满1200个调制符号: 什么也不做, 等下一次调制补齐
}
```

要点

- cur_modulation_result_p：调制输出写指针； cur_mrd_acc_in_p：尚未送 MRD 的读指针。两者差值就是“待送 MRD 的符号数”。
- 满 1200（一个 OFDM 符号的数据量）且 MRD 空闲→触发 mrd_accelerator_genHeadDirect_2045sample_3ddma(...)；随后目的缓冲 tx_data_prepare_p 以 2192 步长前移（2048 点 + 144 CP），读指针前移 1200。
- 计时数组 u32_time_start1[...] 打点做性能统计。

B) MODULE_MODULATION：把编码结果调制成 IQ

```
short modulation_choice = ((coding_result_head_t*)cur_coding_result_head_p)->modulation_choice;
short coding_result_len = ((coding_result_head_t*)cur_coding_result_head_p)->coding_result_len;
short coding_result_int_len = coding_result_len / 18;
if (coding_result_len % 18 != 0) coding_result_int_len++;

int *cur_coding_result = cur_coding_result_head_p + 1; // 跳过“头”1个int

// 解包 bitstream, 再做 QPSK 调制, 写入 cur_modulation_result_p
unpack_Bitstream_1bit_short(coding_result_len, cur_coding_result, v_unpack_result_short);
coding_2_modulator_QPSK_vcu_unpack_Bitstream_1bit_short(
    v_unpack_result_short, coding_result_len, cur_modulation_result_p);

cur_coding_result_head_p = cur_coding_result + coding_result_int_len;

// 前进“调制写指针”: 本块产生的调制符号数 = coding_result_len / (2*(modulation_choice+1))
cur_modulation_result_p += coding_result_len / (2*(modulation_choice+1));

// 继续驱动 MRD 步骤
msg_receive.functionIndex = MODULE_MRD;
osa_queue_send(queue_for_symbol_prss, &msg_receive, OSA_WAIT_FOREVER);
```

要点

- 编码结果布局: [头(1×int)] + [数据(coding_result_int_len×int)]; cur_coding_result_head_p 指向当前块头。
- coding_result_len 是比特数; /18 代表数据区以 18bit/word 的压缩组织（不整除向上取整）。

- QPSK 情况 (modulation_choice=0): 每符号 2 bit, 故输出 IQ 样点数 = coding_result_len / 2 。代码写成通式 coding_result_len / (2 * (modulation_choice + 1)) 。
- 结尾把消息改回 MODULE_MRD 投回队列, 让 A) 分支检查是否已满 1200 并送 MRD。

C) MODULE_SYMBOL_2_BURST : 把符号搬入突发区 (加 CP)

```
if (curIndex % 2) {
    dmag_mem_copy_via_ch7_noISR(symol_to_be_moved_p, symol_to_be_moved_p + 2048, 144*4);
} else {
    dmag_mem_copy_via_ch8_noISR(symol_to_be_moved_p, symol_to_be_moved_p + 2048, 144*4);
}
symol_to_be_moved_p += 2192;
curIndex += 1;
```

要点

- 使用 DMA7/DMA8 (无中断版本) 把 **CP(144×4B)** 从符号尾部搬到前面 (或做相关拼接), 完成“符号→突发”的最终拼装。
- symol_to_be_moved_p 每次按 **2192** 前移 (2048 数据 + 144 CP)。
- curIndex 自增; 计时打点 u32_time_start1[...] 。

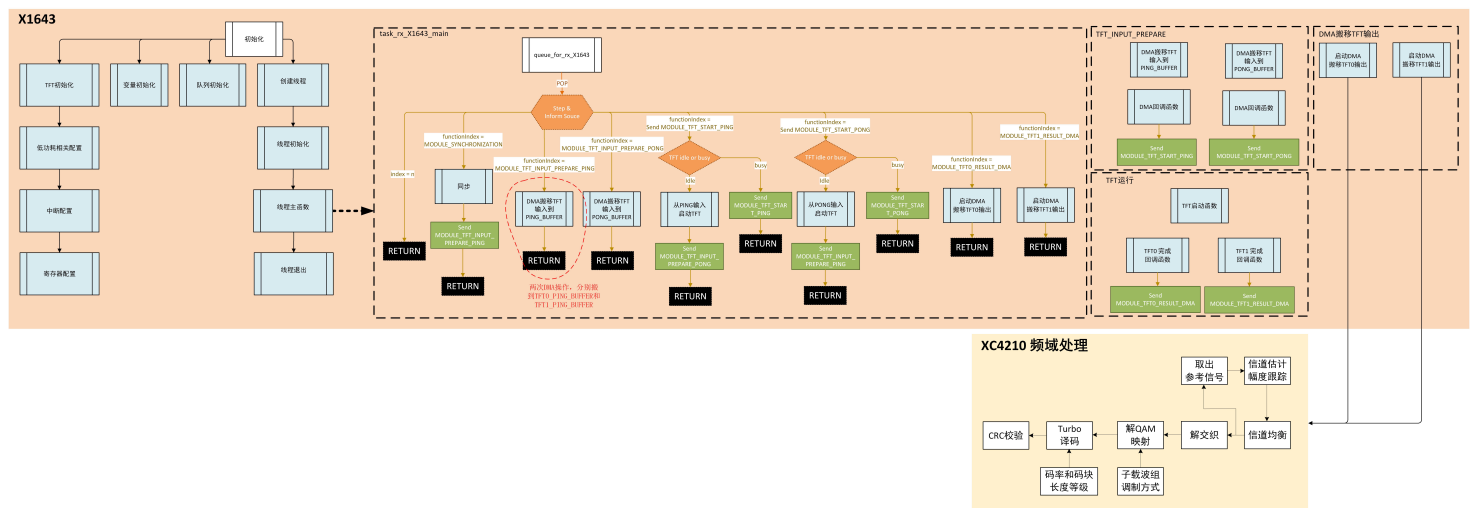
至此 OFDM 的时域数据生成完毕, 数据存储在特定区域, 等待射频发送模块调用。

(c) DMAG 通道6 ISR

在编码结果 DMA 传输完成后 (由通道6触发的中断服务程序调用) 发消息通知符号处理任务: dmag_mem_copy_via_ch6_isr_for_coding_result_from_X1643()

意味着: 编码器结果已经搬运到目标内存, 后续处理可以继续。

七、收端结构



1. X1643 核

(a) 初始化

init_task_queue_for_1643_for_rx() 是 X1643 侧“接收 (RX) 链路”的初始化。

它把队列、接收线程、DMA/TFT 中断、硬件配置都准备好, 并把加速器状态置空闲:

1. 创建接收消息队列 & 线程

- `queue_for_rx = OSA_QUEUE_CREATE("queue_for_rx", 4, 16);`
建一个 RX 用队列。
- `osa_task_create("task_rx", 55, 100, task_rx_init, task_rx_main, task_rx_delete, DEMO_TASK_SPOOL_ID_1, 2048);`
创建接收线程：
 - `task_rx_init` 做资源初始化
 - `task_rx_main` 阻塞收 `queue_for_rx` 的消息并驱动 RX 流水线
 - `task_rx_delete` 释放资源

2. DMAG 初始化 & 中断注册

- `dmag_init_config();`
初始化 DMA (DMAG) 控制器的通道/优先级/触发等基础配置。

3. TFT 相关配置 & 中断注册

• DMA 通道 1 中断:

```
osa_irq_unregister(CP_DMAG_CH1_INT);  
osa_irq_register(CP_DMAG_CH1_INT, dmag_mem_copy_via_ch1_isr, NULL_PTR);
```

当 **ch1 搬移完成**时进入 `dmag_mem_copy_via_ch1_isr`，通常在 ISR 里向 `queue_for_rx` 投消息，唤醒 `task_rx_main` 继续处理。

• TFT (Time-Frequency Transform, 频域/时域转换) 完成中断:

```
osa_irq_unregister(LTE_DL_TFT0_DONE_INT);  
osa_irq_register(LTE_DL_TFT0_DONE_INT, algo_lte_dltft0_done_callback, NULL_PTR);  
  
osa_irq_unregister(LTE_DL_TFT1_DONE_INT);  
osa_irq_register(LTE_DL_TFT1_DONE_INT, algo_lte_dltft1_done_callback, NULL_PTR);
```

两路 TFT 完成回调分别触发 0/1 号流水。

- `rx_tft_demo_config_no_register();`
配置 TFT 的寄存器/参数。

4. 置加速器状态为空闲

- `TFT_acc_idle_or_not = TFT_ACC_IDLE;`
表示 TFT 加速器可用；主线程在触发 TFT 前会检查这个标志，避免重入。

(b) 线程主函数

X1643 侧 RX 线程主循环 (`task_rx_main`)。它从 `queue_for_rx` 队列阻塞取消息，根据 `functionIndex` 驱动 **同步** → **启动TFT** → **准备输入 (PING/PONG)** → **搬出TFT结果** 的流水线，并在多个节点用硬件计时器打点做性能统计。

主循环

- 永久阻塞取消息：`osa_queue_receive(queue_for_rx, ..., OSA_WAIT_FOREVER);`

1) MODULE_SYNCHRONIZATION —— 起始同步

- 同步 (定时/频偏) 算法。
- 同步完成后推进到“准备 TFT 输入 (PING/PONG)”阶段，并设 `TFT_PING_OR_PONG=1`。

2) MODULE_TFT_START —— 启动TFT

```
u32_time_start[40+symbol_TFTin_idx] = *pCP_TIMER3_TCV;

if (TFT_acc_idle_or_not == TFT_ACC_IDLE) {
    ltet_start_from_ping_or_pong(TFT_PING_OR_PONG); // 触发TFT从当前PING/PONG启动
    msg_receive.functionIndex = MODULE_TFT_INPUT_PREPARE_PING_PONG;
    osa_queue_send(queue_for_rx, &msg_receive, OSA_WAIT_FOREVER);
} else {
    // 忙则把消息丢回队列等待稍后重试
    osa_queue_send(queue_for_rx, &msg_receive, OSA_WAIT_FOREVER);
}
```

- 功能：当 TFT 空闲时按当前 PING/PONG 触发一次变换；否则重投消息等待。
- 打点记录每次启动时刻。

3) MODULE_TFT_INPUT_PREPARE_PING_PONG —— 准备TFT输入

```
u32_time_start[20+symbol_TFTin_idx] = *pCP_TIMER3_TCV;

TFT_PING_OR_PONG = (TFT_PING_OR_PONG+1) % 2; // 切换到另一半缓冲
TFT0_INPUT_ADD = HAC_LTE_BASE_ADDR_LTET + 0x00010000 + TFT_PING_OR_PONG*0x00002800;
TFT1_INPUT_ADD = HAC_LTE_BASE_ADDR_LTET + 0x00020000 + TFT_PING_OR_PONG*0x00002800;

// 将两路（tft0/tft1）输入各 2560*4 字节搬入各自 SPRAM
dmag_mem_copy_via_ch0_2_X1643_srcIn_destIn((UINT32_PTR)TFT0_INPUT_ADD, TFT0_DAMD0_Input, 2560*4);
dmag_mem_copy_via_ch1_2_X1643_srcIn_destIn((UINT32_PTR)TFT1_INPUT_ADD, TFT1_DAMD0_Input, 2560*4);

symbol_TFTin_idx += 2;
```

- 作用：为下一次 TFT 变换准备好两路输入（双缓冲 ping-pong），两条 DMA（ch0/ch1）并行搬入 2560 样点（单位按实现为 4B）。
- 每准备完一对（两个符号）的输入，symbol_TFTin_idx += 2。

4) MODULE_TFT1_RESULT_DMA —— 搬运TFT结果到4210侧 SPRAM

```
u32_time_start[3+symbol_idx] = *pCP_TIMER3_TCV;

// 将两路 1200*4 字节结果搬到 4210 显式的 SPRAM 输出区
dmag_mem_copy_via_ch3_for_TFT_result_send_2_XC4210_srcIn_dest4210Out(
    RX_FLOW_TEST_IN32_p, (UINT32_PTR)HAC_LTE_TFT0_ANT0_SYM_SPRAM_ADDR, 1200*4);
dmag_mem_copy_via_ch4_for_TFT_result_send_2_XC4210_srcIn_dest4210Out(
    RX_FLOW_TEST_IN32_p + 1200, (UINT32_PTR)HAC_LTE_TFT1_ANT0_SYM_SPRAM_ADDR, 1200*4);

// 推进输出计数与源指针（一次两符号，总计2400样点）
symbol_idx += 2;
RX_FLOW_TEST_IN32_p += 2400;
```

- 这是把 TFT 计算结果从 DDR（或共享缓存）搬到 4210 的两个 SPRAM 结果区，各 1200 样点（4字节单位）。
- 每次处理两路（两符号），推进 symbol_idx += 2。
- RX_FLOW_TEST_IN32_p 前移 2400，用于下一批结果。

TFT ISR

algo_lte_dltft1_done_callback() :

- 当 TFT1 运算完成 时触发。
- 标记 TFT1 加速器空闲，可接受新任务。
- 构造消息，通知接收线程有新的 TFT1 运算结果 DMA 可以处理。

2. XC4210 核

(a) 初始化

XC4210 侧把“接收（RX）流程”的消息队列、任务线程和 DMA 中断都准备好。

在 **XC4210** 侧把“接收（RX）流程”的消息队列、任务线程和 DMA 中断都准备好。

init_XC4210_task_for_rx()

1. 初始化队列与任务

- init_task_queue_for_4210_for_rx();

2. 注册 DMAG 通道 3、4 的中断回调

- CP_DMAG_CH3_INT 、 CP_DMAG_CH4_INT → 统一回调 dmag_mem_copy_via_ch34_isr_for_coding_result_from_X1643
- **RX 路径里有两条 DMA 通道（3/4）参与数据搬运**，完成时用同一个 ISR 进入“下一步处理”。

3. 启用并放通对应中断源

- *XC4210_ICTL_IRQ_INTEN |= (1<<0); // 允许中断源 0
- *XC4210_ICTL_IRQ_INTMASK &= ~(1<<0); // 不屏蔽中断源 0

init_task_queue_for_4210_for_rx() :

1. 建立接收队列

- queue_for_rx = OSA_QUEUE_CREATE("queue_for_rx", 4, 16);
- 用于 **RX 任务**的消息投递

2. 创建接收线程 task_rx

- osa_task_create("task_rx", 55, 100, task_rx_init_XC4210, task_rx_main_XC4210, task_rx_delete_XC4210, DEMO_TASK_SPOOL_ID_1, 2048);
- init：做接收侧资源初始化
- main：阻塞接收 queue_for_rx 的消息，驱动 RX 流水线（解调/均衡/解码等）
- delete：清理资源

(b) 线程主函数

XC4210 接收链路里的“频域符号处理”线程主函数（task_rx_main_XC4210）。

它从 queue_for_rx 取消息，当收到 MODULE_FREQSYMPROCESS 时，对当前频域符号做信道估计/均衡/频偏与线性插值修正/软解调，并用计时数组打点做性能统计。

主循环：

- 永久阻塞等待：osa_queue_receive(queue_for_rx, ..., OSA_WAIT_FOREVER);
- 仅当 msg_receive.functionIndex == MODULE_FREQSYMPROCESS 才处理，这由前级 **TFT/FFT 完成 + DMA 搬运完成** 的中断发来。

处理逻辑

- sym_freq_idx == 0 （首个符号）：

- **信道估计**： `CDivision_v2_new(freq_sym, Rebuild_ControlSymbol, He);`
用导频/控制符号 `Rebuild_ControlSymbol` 对首符号 `freq_sym` 做 LS/MMSE 类估计，得到信道响应 `He`。
- `sym_freq_idx > 0` （其余符号）：
 - i. **信道均衡**： `ChannelEqualization(freq_sym, He);`
用已估的 `He` 均衡当前符号。
 - ii. **均衡后修正**：
 - `CModifyLScf_V5_new(freq_sym, freq_sym, Rebuild_ControlSymbol, 0);`
基于控制符号的修正/约束（子载波置信度或残余误差校正）。
 - `CModifyLinearInterpolation_odd_V2_new_2(Rebuild_ControlSymbol, freq_sym);`
对导频间隔的子载波做**线性插值**补偿。
 - iii. **频率/多普勒微调**： `fudutiaozhen(freq_sym, kb_list, He);`
结合 `kb_list` （每子载波的补偿系数）做残余频偏/相位漂移校正。
 - iv. **软解调（QPSK）**：
 - 打点： `u32_time_start1[0/1]`
 - `DemodulationQPSK_100sc_opt(freq_sym, llr_sym);` 输出 **LLR** 到 `llr_sym`，供后续译码。
- 处理完成： `sym_freq_idx += 1`；递增当前已处理符号计数。

(c) DMAG ISR

```
dmag_mem_copy_via_ch34_isr_for_coding_result_from_X1643()
```

DMA 通道3和4完成的中断回调函数，通过构造一个消息 `msg_send`，并把它发送到 `queue_for_rx` 队列，通知接收线程执行 `MODULE_FREQSYMPROCESS`（频域符号处理）。

中断 → 消息通知 → 唤醒处理任务

测试流程

2,7777,0,0: XC4210 线程初始化

1,7777,0,0: X1643 线程初始化，触发发射线程

1,7778,0,0: X1643 LOG输出

2,7778,0,0: XC4210 LOG输出

1,6666,0,0: X1643 线程初始化，触发接收处理

1,6667,0,0: X1643 LOG输出

2,6667,0,0: XC4210 LOG输出